



INSTITUT NATIONAL SUPERIEUR DES
SCIENCES ET TECHNIQUES D'ABECHE

CAHIER DES CHARGES Projet Personnelle et Professionnelle

*3ème Année en Génie Informatique
Option Génie Logiciel*

IA pour la conversation de schéma UML en code fonctionnel

Réalisé par :

Abakar Mahmat Brahim

Encadreur (s) :

Encadrant Professionnel : Dr. MAHAMAT HABIB

Encadrant Pédagogique : M. ABOUBAKAR

Année universitaire : 2025/2026

Table des matières

1	Introduction	4
1.1	Contexte général	4
1.2	Présentation du projet	4
1.3	Objectifs du document	4
2	Analyse de l’Existant	5
2.1	Outils existants	5
2.2	Valeur ajoutée du projet	5
3	Périmètre du Projet	5
3.1	Ce qui est inclus	5
3.2	Ce qui est exclu	6
4	Acteurs et Cas d’Utilisation	7
4.1	Acteurs identifiés	7
4.2	Cas d’utilisation principaux	7
4.3	Diagramme de Cas d’Utilisation	8
5	Exigences Fonctionnelles	8
5.1	Module Upload	8
5.2	Module Parsing	9
5.3	Module Génération IA	9
5.4	Module Interface et Expérience Utilisateur	10
6	Exigences Non Fonctionnelles	11
7	Architecture Technique	12
7.1	Stack technologique	12
7.2	Architecture globale	12
7.3	Diagramme de Classes	14
7.4	Diagramme de Séquence	15
7.5	Structure des dossiers	16
8	Cycle de Vie du Projet	16
8.1	Choix du cycle de vie : Modèle Incrémental	16
8.2	Comparaison des cycles de vie	16
8.3	Les 5 incréments du projet	17
8.4	Jalons critiques	17
9	Planning de Réalisation	18

10 Livrables Attendus	19
11 Critères d'Acceptation	19
11.1 Critères techniques	19
11.2 Critères académiques	20
12 Glossaire	20

1. Introduction

1.1 Contexte général

Dans le domaine du Génie Logiciel, la modélisation UML (*Unified Modeling Language*) constitue une étape fondamentale du cycle de développement. Elle permet de concevoir et de documenter l'architecture d'un système avant son implémentation. Cependant, le passage du modèle conceptuel au code source reste une tâche manuelle, répétitive et source d'erreurs.

Ce projet s'inscrit dans le paradigme du **Développement Dirigé par les Modèles** (MDD — *Model-Driven Development*), qui préconise la génération automatique de code à partir de modèles de haut niveau. L'intégration de l'Intelligence Artificielle générative dans ce processus représente une avancée significative, permettant de produire un code plus complet, contextuel et directement exploitable.

1.2 Présentation du projet

Le projet « **UML-to-Code** » vise à développer une application web full-stack permettant de transformer automatiquement un diagramme de classes UML en code source fonctionnel, en exploitant le modèle d'IA générative **Groq Llama 3.3-70b**. L'utilisateur soumet un diagramme (au format XML, image ou PDF), et l'application génère le code Python ou PHP correspondant, ainsi qu'un serveur Flask complet et opérationnel, le tout exportable en archive ZIP.

Changement majeur (v1 → v2) : Le moteur d'IA a été migré de *Google Gemma via API Gemini* vers **Groq Llama 3.3-70b (API Groq)**, offrant des temps de génération réduits (2 secondes) et un code de meilleure qualité. L'interface a également été entièrement enrichie.

1.3 Objectifs du document

Ce Cahier des Charges a pour objectifs de :

- Définir le périmètre fonctionnel et technique du projet
- Identifier les acteurs et leurs interactions avec le système
- Spécifier les exigences fonctionnelles et non fonctionnelles
- Présenter l'architecture technique retenue
- Établir le planning de réalisation
- Définir les critères d'acceptation et les livrables attendus

2. Analyse de l’Existant

2.1 Outils existants

Plusieurs outils de transformation UML vers code existent sur le marché. Une analyse comparative permet d’identifier leurs limites :

Outil	Langages générés	Format entrée	Limites principales
Acceleo (Eclipse)	Java, Python, PHP	XMI/EMF	Très complexe, lourd à installer
AndroMDA	Java EE uniquement	XMI	Limité à Java, pas d’IA
GenMyModel	Java, Python	En ligne	Payant, pas open source
StarUML Plugins	JavaScript, Java	.mdj	Limité, pas d’IA générative
Solutions manuelles	Tous langages	Tous formats	Lent, source d’erreurs

TABLE 1 – Comparatif des outils existants de transformation UML vers code

2.2 Valeur ajoutée du projet

Par rapport aux solutions existantes, UML-to-Code apporte :

- Intégration d’une IA générative (**Groq Llama 3.3-70b**) pour un code contextuel et complet
- Support de plusieurs formats d’entrée : XMI, image PNG/JPG, PDF
- Génération automatique d’un serveur Flask complet et opérationnel
- Interface web moderne avec design **Glassmorphism**, thème sombre/clair
- Éditeur de code intégré (**Monaco Editor** — le moteur de VS Code)
- Gestion de l’historique local des 10 dernières générations
- Export en archive ZIP (code + serveur Flask)
- Solution légère, open source et sans installation complexe

3. Périmètre du Projet

3.1 Ce qui est inclus

- Acceptation de diagrammes UML au format XMI (StarUML)
- Acceptation de diagrammes sous forme d’image (PNG, JPG)

- Acceptation de diagrammes intégrés dans un fichier PDF
- Génération de code Python (classes, attributs, méthodes, héritage)
- Génération de code PHP (classes avec visibilité, constructeur)
- Génération automatique d'un serveur Flask Python complet avec routes CRUD
- Interface web React.js (Glassmorphism) avec upload, visualisation et téléchargement
- Intégration de **Monaco Editor** pour la lecture et l'édition du code généré
- Système d'historique local (localStorage) — 10 dernières générations
- Mode sombre / mode clair (bascule dynamique)
- Barre de progression en temps réel avec étapes détaillées
- Aperçu visuel du diagramme uploadé (pour PNG/JPG)
- Statistiques de génération : lignes, classes, méthodes, temps
- Notifications toast (feedback utilisateur)
- Export ZIP du projet complet (code + serveur Flask)
- Copie du code généré en un clic
- Téléchargement du code généré en fichier `.py` ou `.php`
- Navbar responsive avec menu hamburger mobile

3.2 Ce qui est exclu

- Les diagrammes de séquence, d'activité ou de cas d'utilisation
- La génération de bases de données ou de requêtes SQL
- La génération de code Java (hors scope pour ce projet)
- L'authentification et la gestion multi-utilisateurs centralisée
- La sauvegarde serveur (historique en localStorage uniquement)

4. Acteurs et Cas d'Utilisation

4.1 Acteurs identifiés

Acteur	Rôle	Interactions principales
Utilisateur	Étudiant ou développeur	Upload fichier, choix options, génération, visualisation Monaco Editor, copie/téléchargement, consultation historique, bascule thème
API Groq (IA)	Modèle Llama 3.3-70b	Reçoit la description UML extraite, retourne le code généré
Backend Flask	Serveur Python	Orchestration, parsing XMI/image/PDF, appel API Groq, génération ZIP
localStorage	Stockage navigateur	Persiste l'historique des 10 dernières générations et le thème

TABLE 2 – Acteurs du système UML-to-Code

4.2 Cas d'utilisation principaux

1. L'utilisateur uploade un fichier XMI, image PNG/JPG ou PDF (drag-and-drop ou clic)
2. Si image PNG/JPG : le système affiche un aperçu visuel du diagramme
3. L'utilisateur choisit le langage cible (Python ou PHP) et le mode (Algorithmique ou IA Groq)
4. La barre de progression affiche les étapes en temps réel
5. Le système détecte le format, parse le diagramme et envoie à Groq si mode IA
6. Le code généré s'affiche dans le Monaco Editor (onglet Code + onglet Serveur Flask)
7. Les statistiques de génération sont affichées (lignes, classes, temps)
8. L'utilisateur copie, télécharge en `.py/.php`, ou exporte en ZIP
9. La génération est sauvegardée dans l'historique local (max 10 entrées)
10. L'utilisateur peut consulter et recharger ses anciennes générations via l'historique

4.3 Diagramme de Cas d'Utilisation

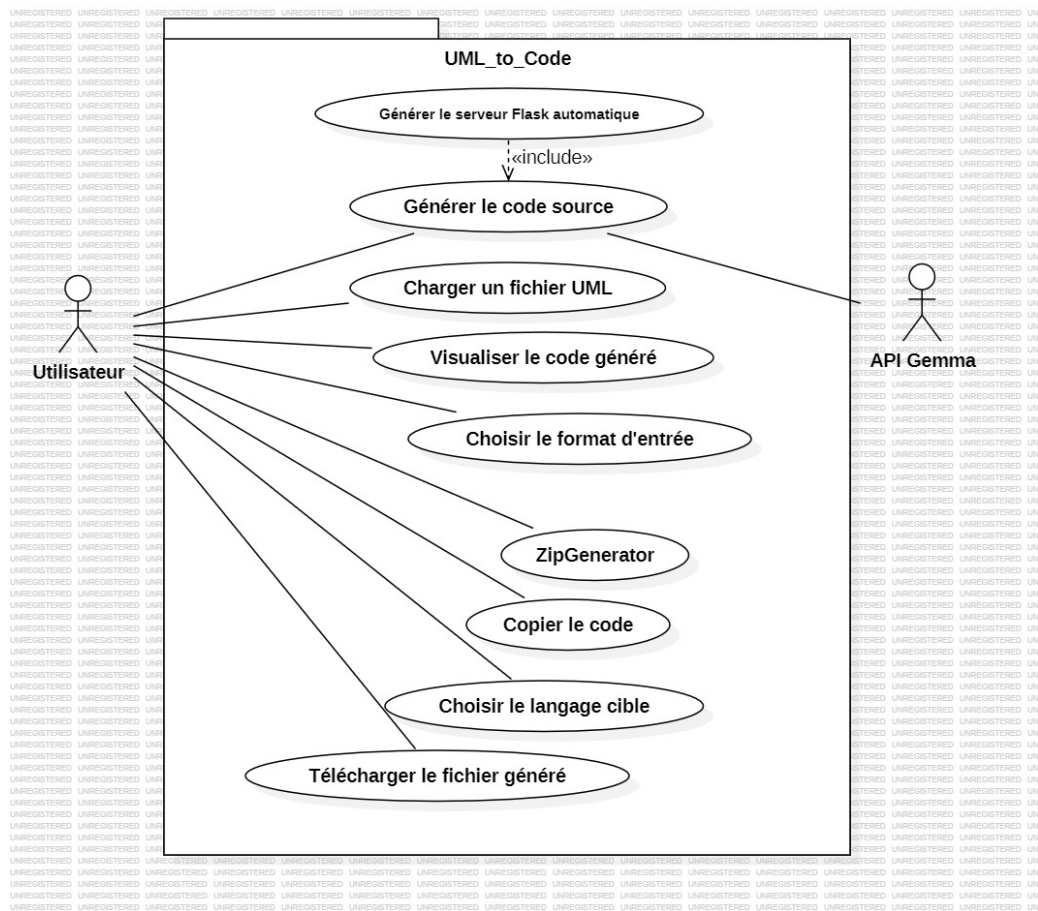


FIGURE 1 – Diagramme de cas d'utilisation — UML-to-Code

5. Exigences Fonctionnelles

5.1 Module Upload

ID	Exigence	Priorité
EF-01	Accepter les fichiers XMI exportés depuis StarUML	Haute
EF-02	Accepter les images PNG et JPG de diagrammes UML	Haute
EF-03	Accepter les fichiers PDF contenant un diagramme UML	Moyenne
EF-04	Valider le format du fichier avant traitement	Haute
EF-05	Afficher un message d'erreur en cas de format invalide	Haute
EF-06	Afficher un aperçu visuel du diagramme pour PNG/JPG	Moyenne
EF-07	Permettre la suppression du fichier sélectionné avant génération	Moyenne

TABLE 3 – Exigences fonctionnelles — Module Upload

5.2 Module Parsing

ID	Exigence	Priorité
EF-08	Parser les classes, attributs et méthodes depuis un XMI	Haute
EF-09	Extraire les relations d'héritage entre classes	Haute
EF-10	Extraire les associations et agrégations	Moyenne
EF-11	Envoyer l'image/PDF directement à Groq (vision IA)	Haute

TABLE 4 – Exigences fonctionnelles — Module Parsing

5.3 Module Génération IA

ID	Exigence	Priorité
EF-12	Appeler l'API Groq (Llama 3.3-70b) avec la description UML extraite	Haute
EF-13	Générer le code Python complet avec <code>__init__</code> , getters, setters	Haute
EF-14	Générer le code PHP avec visibilité, constructeur, getters	Haute
EF-15	Générer un serveur Flask automatique avec les routes CRUD	Haute
EF-16	Permettre le choix du langage cible : Python ou PHP	Haute
EF-17	Permettre le choix du mode : Algorithmique (XMI) ou IA Groq	Haute
EF-18	Afficher les statistiques de génération (lignes, classes, méthodes, temps)	Moyenne

TABLE 5 – Exigences fonctionnelles — Module Génération IA

5.4 Module Interface et Expérience Utilisateur

ID	Exigence	Priorité
EF-19	Afficher le code généré dans Monaco Editor (VS Code intégré)	Haute
EF-20	Afficher onglet Code et onglet Serveur Flask séparément	Haute
EF-21	Permettre la copie du code en un clic	Haute
EF-22	Permettre le téléchargement du fichier .py ou .php	Haute
EF-23	Permettre l'export en archive ZIP (code + serveur Flask)	Haute
EF-24	Afficher une barre de progression avec étapes détaillées en temps réel	Haute
EF-25	Implémenter un historique local (localStorage) des 10 dernières générations	Haute
EF-26	Permettre de recharger une génération passée depuis l'historique	Haute
EF-27	Permettre d'effacer l'historique local	Moyenne
EF-28	Fournir une interface premium type "Glassmorphism"	Moyenne
EF-29	Implémenter le mode sombre / mode clair (persisté en localStorage)	Moyenne
EF-30	Afficher des notifications toast (succès, erreur, info)	Moyenne
EF-31	Navbar responsive avec menu hamburger pour mobile	Moyenne
EF-32	Animations et transitions modernes (fadeIn, float, slideUp)	Basse

TABLE 6 – Exigences fonctionnelles — Module Interface UX

6. Exigences Non Fonctionnelles

Catégorie	Exigence	Critère mesurable
Performance	Temps de réponse de la génération	< 15 secondes par diagramme
Disponibilité	Fonctionne en local	Sans connexion (sauf API Groq)
Fiabilité	Taux de succès de la génération	> 90% sur les cas de test
Maintenabilité	Code source documenté et structuré	Commentaires sur chaque fonction
Portabilité	Fonctionne sur Windows et Linux	Testé sur les deux systèmes
Sécurité	Clé API stockée en variable d'environnement	Fichier <code>.env</code> non versionné (gitignore)
Utilisabilité	Interface intuitive sans formation	Prise en main < 5 minutes
Responsivité	Interface adaptée mobile et desktop	Testé sur écrans 320px à 1440px

TABLE 7 – Exigences non fonctionnelles

7. Architecture Technique

7.1 Stack technologique

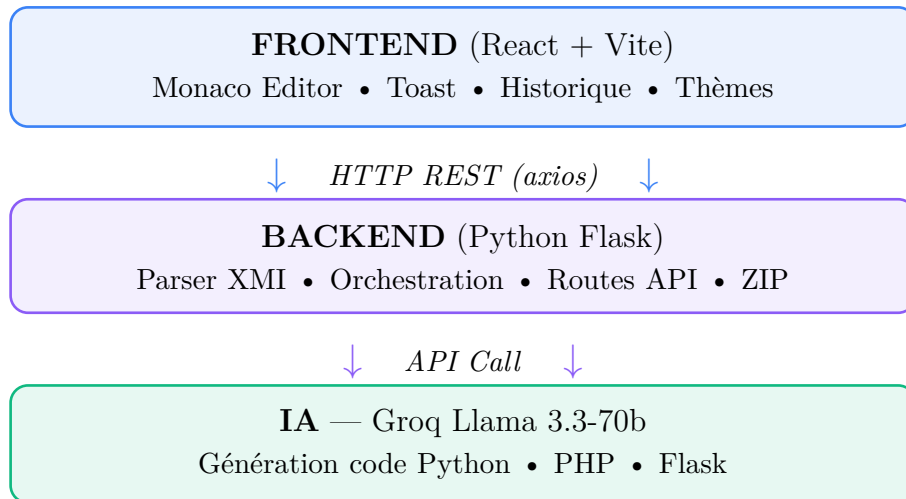
Couche	Technologie	Version	Rôle
Frontend	React.js + Vite	6.x	Interface utilisateur web
Éditeur	Monaco Editor	Latest	Éditeur de code intégré (VS Code)
Icônes	React Icons	Latest	Icônes UI (FaHome, FaBolt, etc.)
HTTP Client	Axios	Latest	Requêtes Frontend → Backend
Backend	Python Flask	3.0.0	Serveur API REST
IA	Groq Llama 3.3-70b	Latest	Génération de code intelligent
Parsing XMI	xml.etree	Natif Python	Extraction classes UML depuis XMI
Parsing Image	PyMuPDF	1.27	Conversion PDF → image
Export	zipfile	Natif Python	Export projet ZIP
Stockage	localStorage	Natif Web	Historique local + thème
Versioning	Git + GitHub	—	Gestion du code source

TABLE 8 – Stack technologique du projet

7.2 Architecture globale

Le système suit une architecture client-serveur en 3 couches :

1. **Couche Présentation** : React.js — interface Glassmorphism, Monaco Editor, historique localStorage, thème sombre/clair, animations
2. **Couche Métier** : Flask Python — orchestration, parsing XMI/image/PDF, appel API Groq, génération ZIP
3. **Couche IA** : API Groq (Llama 3.3-70b) — génération intelligente du code Python/PHP/Flask



7.3 Diagramme de Classes

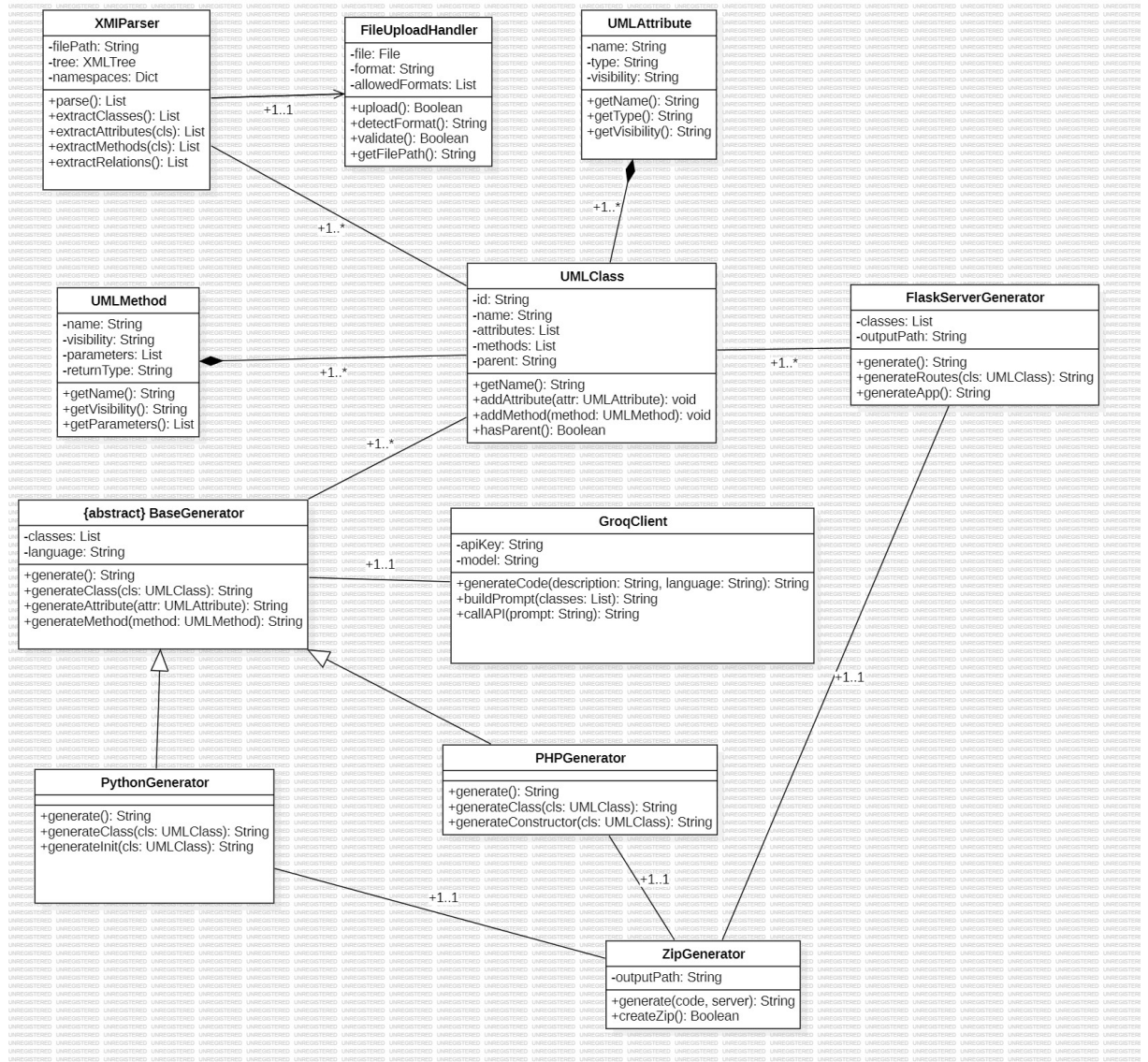


FIGURE 2 – Diagramme de classes — UML-to-Code

7.4 Diagramme de Séquence

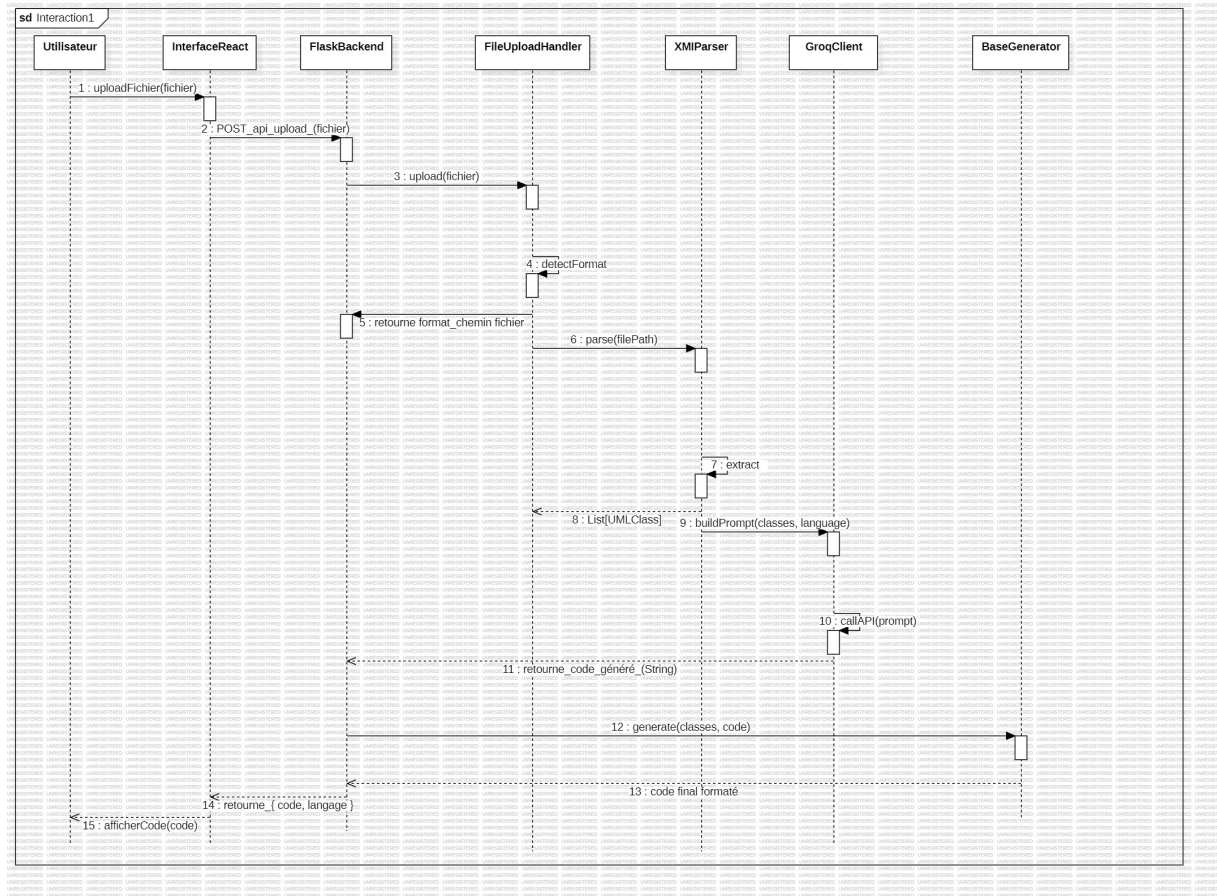


FIGURE 3 – Diagramme de séquence — Génération de code

7.5 Structure des dossiers

```

uml-to-code/
  backend/      # Serveur Python Flask
    app.py      # Point d'entrée Flask
    routes/     # Endpoints API
    parser/xml  # Parser fichiers XMI
    parser/image_analyse/page/PDF via IA
    ai/gemma_#llm_infer_API Groq
    generator/#GénérateurPython
    generator/#GénérateurPHP
    generator/#GénérateurFlaskpy
    generator/#GénérateurZAP.py
  frontend/    # Interface React + Vite
    src/App.js  # Composant principal
    src/App.css # Styles complets
  examples/testFixme.xmi # Fichier XMI de test
  .gitignore
  README.md

```

8. Cycle de Vie du Projet

8.1 Choix du cycle de vie : Modèle Incrémental

Le modèle de cycle de vie retenu pour ce projet est le **modèle incrémental**. Ce choix est motivé par trois raisons principales :

- La durée courte du projet (3 semaines) nécessite des livraisons rapides et concrètes
- Le projet est réalisé en solo, ce qui exclut les cycles nécessitant une équipe (Scrum)
- L'intégration d'une IA générative (Groq) introduit des incertitudes techniques qui imposent un ajustement progressif

8.2 Comparaison des cycles de vie

Cycle de vie	Durée idéale	Solo	3 semaines	Décision
Cascade (Waterfall)	6+ mois	Oui	Non	Rejeté
Cycle en V	3+ mois	Oui	Limite	Rejeté
Agile / Scrum	Continu	Limite	Oui	Rejeté
Incrémental	1–6 mois	Oui	Oui	RETENU

TABLE 9 – Comparaison des cycles de vie

8.3 Les 5 incréments du projet

Incrément	Période	Fonctionnalités ajoutées	Version
0 — Préparation	05–07 mai	Questions prof, étude XMI, recherche, CDC	v0.0
1 — Fondations	08–14 mai	Architecture, parser XMI, modèle de données, test API Groq	v0.1
2 — Cœur IA & UI	15–21 mai	Groq complet, générateurs Python+PHP, Flask auto, React Glassmorphism, Monaco Editor, Historique local	v0.2
3 — Finalisation	22–26 mai	Tests, corrections, export ZIP, thème sombre/clair, animations, rapport, slides	v1.0
4 — Améliorations	27 mai+	Optimisations UI, statistiques, toast, hamburger, corrections bugs	v1.1

TABLE 10 – Les 5 incréments du projet

8.4 Jalons critiques

Date	Jalon	Critère de validation
07/05/2026	Jalon 0 — Lancement	CDC validé, questions prof répondues
14/05/2026	Jalon 1 — v0.1	Parser XMI fonctionnel, API Groq testée
21/05/2026	Jalon 2 — v0.2	Application complète fonctionnelle end-to-end
24/05/2026	Jalon 3 — v1.0	Rapport rendu, slides prêtes
26/05/2026	Jalon 4 — Soutenance	Démo réussie devant le jury

TABLE 11 – Jalons critiques du projet

9. Planning de Réalisation

Semaine	Période	Tâches principales	Livrables
S0	05–07 mai	Questions prof, étude XMI, recherche	Questions validées
S1	08–14 mai	CDC, diagrammes UML, architecture, setup projet	CDC + diagrammes
S2	15–21 mai	Parser XMI, API Groq, Monaco Editor, Historique, UI	Application complète
S3	22–26 mai	Tests, rapport, slides, export ZIP, répétition soutenance	Rapport + soutenance

TABLE 12 – Planning de réalisation

10. Livrables Attendus

#	Livrable	Format	Date cible
1	Cahier des Charges (ce document)	Word + PDF	09/05/2026
2	Diagrammes UML de conception (Use Case, Classes, Séquence)	StarUML (.mdj)	11/05/2026
3	Code source complet (backend + frontend)	GitHub	21/05/2026
4	Rapport de projet complet	Word + PDF	24/05/2026
5	Présentation slides	PowerPoint/PDF	24/05/2026
6	Démonstration live	Application hébergée — uml-to-code-peach.vercel.app	26/05/2026

TABLE 13 – Livrables attendus

11. Critères d'Acceptation

11.1 Critères techniques

- L'application accepte au moins 3 formats d'entrée (XMI, image PNG/JPG, PDF)
- Le code Python généré est syntaxiquement correct et exécutable
- Le code PHP généré respecte la syntaxe PHP 8+
- Le serveur Flask généré contient les routes CRUD de base
- L'interface React est fonctionnelle et responsive (mobile + desktop)
- L'application est déployée et accessible en ligne sur uml-to-code-peach.vercel.app
- Le Monaco Editor affiche le code avec coloration syntaxique

- L'historique local sauvegarde et restaure les 10 dernières générations
- L'export ZIP contient le code + le serveur Flask, prêt à déployer
- Le mode sombre / clair est fonctionnel et persisté
- Le temps de génération est inférieur à 15 secondes

11.2 Critères académiques

- Le code source est versionné sur GitHub avec des commits réguliers
- Le rapport suit un plan académique complet (8 chapitres minimum)
- La démo fonctionne sans erreur devant le jury
- L'étudiant maîtrise et explique son architecture technique

12. Glossaire

Terme	Définition
UML	Unified Modeling Language — Langage de modélisation unifié
XMI	XML Metadata Interchange — Format d'échange de modèles UML
MDD	Model-Driven Development — Développement dirigé par les modèles
LLM	Large Language Model — Grand modèle de langage (ex : Llama 3.3)
API	Application Programming Interface — Interface de programmation
Groq	Service d'inférence IA ultra-rapide utilisant des puces LPU spécialisées
Llama 3.3-70b	Modèle d'IA open source de Meta, 70 milliards de paramètres, hébergé sur Groq
Flask	Micro-framework web Python pour créer des API REST
React.js	Bibliothèque JavaScript pour créer des interfaces utilisateur web
Monaco Editor	Éditeur de code en ligne (le moteur de VS Code) intégré au projet
CRUD	Create, Read, Update, Delete — Opérations de base sur les données
REST	Representational State Transfer — Style d'architecture pour les API web
Glassmorphism	Style de design UI basé sur des effets de transparence et de flou
localStorage	API du navigateur pour persister des données côté client

TABLE 14 – Glossaire du projet

Document rédigé par : **Abakar Mahamat Brahim (ABVIP)** — Étudiant GL3, INSTA Abéché — Mai 2026

GitHub : github.com/ABAKAR5/uml-to-code | Version 2.0